PULSEPOINT

Joint Project Roadmap & Engineering Playbook

A phase-wise plan for two student founders: AI/ML + Full-Stack

WHERE IDEAS MEET THEIR PERFECT TEAM.

Product strategy • Roles and responsibilities • Architecture • AI matching • Repository workflow • Testing • Launch

PROJECT ROLE	PRIMARY OWNERSHIP
Founder A — AI/ML Developer	Matching algorithm, data/feature design, Gemini integration, evaluation, AI quality and guardrails.
Founder B — Full-Stack Developer	Product UI, authentication, database, backend/API, realtime workflows, deployment and app reliability.
Shared ownership	Product decisions, UX, security, testing, code review, demos, documentation and release decisions.

Planning assumption

A 12-week MVP plan for two developers working part-time, approximately 8–12 focused hours per person per week. If exams, college events or other commitments reduce capacity, keep the scope fixed and extend dates instead of cutting security or testing.

Prepared: 29 October 2026 | Status: proposed baseline, to be agreed by both founders

How to use this report

Treat this as the team's initial source of truth. In Week 1, both founders should edit the assumptions, agree on the MVP, and convert every phase into GitHub Issues. Revisit scope weekly; do not silently expand the MVP.

Contents

- 1. Executive summary and product definition
- 2. Product scope: MVP vs. later releases
- 3. Founder roles and ownership boundaries
- 4. Twelve-week phase-wise roadmap
- 5. Recommended architecture and technology stack
- 6. AI matching, PulseIndex and AI mentor design
- 7. Data model and security boundaries
- 8. Repository structure and setup commands
- 9. Git workflow, pull requests and collaboration rules
- 10. Testing, observability and deployment
- 11. Privacy, safety, risks and mitigation
- 12. Launch criteria, metrics and next actions
- Appendix A. Definition of Done and acceptance checklist
- Appendix B. Official technical references

1. Executive summary and product definition

The product in one sentence

PulsePoint helps students discover compatible collaborators, form balanced project teams, find relevant hackathons, and move from idea to execution with practical AI assistance.

The recommended restart is a focused web application—not a collection of AI demos. Start with reliable student profiles, team creation/joining, hackathon discovery, and explainable teammate recommendations. Add AI features only where they improve a measurable user outcome.

The problem to solve

- Students often have ideas but cannot quickly find people with complementary skills, suitable availability, or aligned project goals.
- Hackathon participation is fragmented across social posts, spreadsheets, forms and private groups.
- Teams can form but still struggle to make responsibilities, progress and contribution expectations visible.
- Generic AI chat is easy to copy; a trusted campus workflow and a useful matching loop are the product's differentiators.

Primary users

User	Job to be done	MVP value
Student seeking a team	Find an opportunity or a team where their skills fit.	A trustworthy profile, clear match reasons, and a low-friction request to join.
Student with an idea	Build a team with missing skills and aligned interests.	Create a team, list roles needed, review applicants and invite members.
Hackathon participant	Discover relevant events and organize participation.	Search/filter listings, save an event, and connect it to a team.
Team member / lead	Know what the team is doing next.	Basic team space, task/progress updates and clear roles.

Product principles

- Explain recommendations. Every suggested match should have human-readable reasons, not just a number.
- Opt-in and user-controlled. Ask users for collaboration preferences; do not infer sensitive traits or covertly score personality.
- AI is assistive, not authoritative. Let users correct profiles, reject matches, report abuse and disable AI-generated suggestions.
- Build for fallback. Core browsing and team workflows must still work when the AI provider is rate-limited or unavailable.
- Earn trust before growth. Secure access, accurate listings and useful matches matter more than a long feature list.

2. Product scope: MVP vs. later releases

A two-person team wins by finishing one coherent loop. Do not try to build every feature in the first release.

MVP — must ship

Capability	MVP behavior	Acceptance signal
Authentication	Sign up/sign in, sign out, protected app routes; Google provider if configured.	A new user can create an account and cannot view private app routes while signed out.
Student profile	Name/display name, college, degree/year, skills, interests, goals, available roles, optional links and availability.	Profile can be saved, edited and marked complete; private fields are not public.
Teams	Create a team, define project/skills needed, invite/request to join, accept/reject requests, leave team.	Membership and roles remain consistent after refresh; duplicate membership is prevented.
Discovery	Browse students/teams/hackathons; filters for skills, domain, event date and team status.	Useful results, empty states, loading states and pagination/limits exist.
Match suggestions	Rule-based score plus reasons; filter out blocked, ineligible or already-full matches.	Each score can be explained from visible signals and is tested on fixtures.
Hackathons	Curated/manual event listings, detail page, external registration link, save/bookmark.	Each listing has source, deadline, location/format and last-verified date.
Team workspace	Member list, role ownership, simple task/progress status, optional lightweight chat.	A team can coordinate without depending on AI.
Trust and support	Report user/listing, basic moderation/admin process, privacy notice and delete-account process.	Reports reach an owner and are not silently discarded.

Post-MVP — defer until evidence supports it

- Semantic skill/profile matching with embeddings, after the rule-based baseline has evaluation data.
- AI mentor for project planning, blockers and learning resources, with clear limitations and rate limits.
- AI-assisted profile/tag extraction from text, always requiring user confirmation before saving.
- Advanced analytics, team health trends, internationalization and multi-campus administration.
- Calendar integrations, automated event ingestion, mobile apps, recommendations learned from outcome data.

Cut line if the schedule slips

Keep authentication, profiles, team formation and explainable matching. Reduce rich chat, analytics and AI mentor scope first. Never cut authorization checks, Firestore rules, secret management, backups/export planning or core regression tests.

3. Founder roles and ownership boundaries

Each founder has a primary lane, but both are accountable for the product. Ownership is a default for execution—not a reason to ignore defects outside one lane.

Workstream	AI/ML developer (you)	Full-stack developer (friend)	Shared checkpoint
Product and discovery	Define matching hypotheses, candidate features and evaluation questions.	Turn journeys into screens, API needs and a workable data model.	Both interview 5–8 target students and agree on the top 3 pain points.
UI / UX	Review match explanation, onboarding fields and AI output usability.	Own design system, pages, responsive layouts, forms and loading/error states.	Both approve flows before feature implementation.
Auth / data / API	Specify data needed for matching and least-privilege access.	Own Firebase setup, CRUD flows, server routes, validation and database rules.	Pair-review data contract and authorization-sensitive code.
Matching engine	Own scoring features, weights, explanations, fixture set and evaluation.	Own integration, query constraints, API response types and UI display.	Review algorithm version, failure behavior and product metrics.
Gemini / AI features	Own prompts, schemas, model selection, quality tests, safety and cost controls.	Own secure server-side invocation, loading/error states, timeouts and rate limits.	No API key in browser; review every AI feature before release.
Quality	Own ranking evaluation set, adversarial/edge-case tests and model regression.	Own component/unit tests, integration, E2E, deployment checks and error logging.	Test critical journeys together on every release candidate.
Release / operations	Track AI usage, output quality and user feedback.	Own preview/production deployment, rollback and environment setup.	Both approve production release and review incidents.
Documentation	Document scoring, prompts, assumptions and evaluation results.	Document setup, architecture, API contracts and runbooks.	Update docs in the same PR as behavior changes.

Ownership rules

- One owner per issue. The other person is reviewer or collaborator; “both own it” must not mean nobody is accountable.
- The author tests their change before opening a pull request. The reviewer checks behavior, security and maintainability—not only formatting.
- Every shared contract (TypeScript type, API request/response, Firestore document shape) is agreed before the two workstreams proceed in parallel.
- Any change to scoring weights, data collection, security rules or secrets requires explicit review from both founders.
- Write down decisions that affect architecture in short Architecture Decision Records (ADRs) under docs/decisions/.

4. Twelve-week phase-wise roadmap

This plan assumes part-time capacity of roughly 8–12 focused hours per founder each week. Estimates are planning aids; phase exits are defined by working software and acceptance checks, not by the calendar alone.

Week / phase	Focus and key tasks	AI/ML owner	Full-stack owner	Exit / demo
W1 — Phase 0: Align	Problem interviews, persona, MVP, user flows, acceptance criteria, risks, name/branding decision.	Define matching problem and success metrics.	Draft app flows and technical constraints.	One-page PRD, 5–8 user interviews planned/completed, scope frozen.
W2 — Phase 1: Foundation	Create repo, branch rules, Next.js app, lint/typecheck/test scripts, Firebase dev project, preview deploy.	Add AI module interface and test fixtures; no production model calls yet.	Bootstrap app, auth config, UI shell, CI, Firebase environments.	A clean clone runs; CI passes; preview URL works.
W3 — Phase 2: Identity	Sign-in, onboarding, profile create/edit, profile privacy and validation.	Define skill taxonomy and normalized profile fields.	Implement auth, profile pages, Firestore reads/writes and security rules.	User can sign in and save/edit a profile; rules tested.
W4 — Phase 3: Teams	Team create/edit, skills needed, invites/requests, member roles and status.	Define complementary-skill features and fixture profiles.	Implement team/member schemas and workflows.	Two test accounts can form a team without duplicate membership.
W5 — Phase 4: Match v1	Deterministic scoring, match reasons, filters, score tests and feedback capture.	Implement score function, baseline weights, explanations and offline evaluation.	Add match endpoint/query path, response types and match cards.	Every score is reproducible; fixtures cover edge cases; no AI dependency.

Week / phase	Focus and key tasks	AI/ML owner	Full-stack owner	Exit / demo
W6 — Phase 5: Opportunities	Hackathon listings/detail, search, filters, bookmarks, source and verification date.	Define skill/domain tags shared between profiles and events.	Build listing schema, CRUD/admin seed flow and discovery UI.	Students can find an event and open its official registration link.
W7 — Phase 6: Collaboration	Team space, role/availability visibility, task status and basic notifications/chat if capacity allows.	Review signals used in PulseIndex; ensure opt-in and no surveillance metrics.	Implement activity UI, rules, unread/loading/error states.	Team can agree on roles and show next actions.
W8 — Phase 7: AI assist	Add one bounded Gemini feature: match explanation enhancement OR project mentor prototype.	Own prompt, schema, quality dataset, safety and cost/latency tracking.	Wire server-side call, auth, timeout, rate limiting and UI fallback.	Feature passes quality checks and degrades gracefully when disabled.
W9 — Phase 8: Hardening	Mobile/responsive fixes, accessibility, validation, authorization review, abuse/report flow, telemetry.	Run failure/adversarial tests and scoring audit.	Run E2E, inspect Firestore rules, performance and deploy logs.	Critical journeys pass on fresh accounts; no known critical security bugs.
W10 — Phase 9: Beta	Invite 10–20 students, observe onboarding and team matching, collect feedback.	Measure match relevance, explanation usefulness and false positives.	Capture funnel, errors and task completion; fix highest-friction bugs.	At least 5 real user sessions observed; prioritized feedback backlog.
W11 — Phase 10: Release candidate	Fix P0/P1 issues, confirm privacy/deletion flows, curate first event list and release checklist.	Freeze scoring version; document known limitations and AI cost caps.	Verify prod envs, rollback plan, rules, monitoring and backups/export.	Both founders approve release candidate.
W12 — Phase 11: Launch	Soft launch to one campus/community, gather feedback, ship only safe fixes, retrospective.	Review match feedback and determine next experiment.	Monitor uptime/errors, deployment and support queue.	Public beta note, baseline metrics and next 4-week priorities.

Weekly cadence

Cadence	Timebox	Output
Monday / planning	20–30 min together	Pick 2–4 realistic issues total; agree on owner and acceptance criteria.
Midweek async check	5–10 min per person	Post what shipped, what is blocked, what needs review.
End-of-week demo	30–45 min together	Demonstrate a working path; record bugs and decisions; update roadmap.
Every PR	Review as needed	Automated checks pass, reviewer approves, preview is tested before merge.
Every two weeks	30 min retro	Identify one process improvement; do not add scope without trading something off.

5. Recommended architecture and technology stack

Use a TypeScript-first monolith for the MVP. A single Next.js application plus Firebase services keeps operations manageable for two developers. Do not add a separate Python microservice, vector database, queue or microservice architecture until measured need justifies it.

Layer	Recommendation	Reason / boundary
Web application	Next.js App Router + React + TypeScript	One codebase for routes, layouts, server components and API route handlers. Keep client components limited to interaction-heavy UI.
UI system	Tailwind CSS + shadcn/ui + Lucide; Framer Motion sparingly	Consistent, accessible components and fast iteration. Animation must not block core UX.
Authentication	Firebase Authentication	Use the Firebase SDK for client sign-in. Protect routes and validate identity on server endpoints.
Primary database	Cloud Firestore	Good fit for profile/team documents and realtime team activity. Model around actual queries; Firestore is not a relational database.

Layer	Recommendation	Reason / boundary
File uploads	Cloud Storage for Firebase (only if needed)	Use upload limits, content-type checks and storage rules. Keep uploads out of MVP if links suffice.
Server endpoints	Next.js Route Handlers for MVP	Place privileged operations and AI calls on the server. Add Cloud Functions later for scheduled/background jobs if needed.
AI provider	Gemini API via official Google GenAI SDK, server-side only	Use schema-constrained outputs where useful, validate semantic correctness, track latency/cost, and implement provider failure fallback.
Validation	Zod or equivalent schema validation	Validate browser inputs and server boundaries; TypeScript types alone do not validate incoming runtime data.
Hosting	Vercel for Next.js; Firebase for auth/data/storage	Use separate local, preview and production configuration. Test preview deployments before production.
Source control / CI	GitHub + pull requests + GitHub Actions	Require lint, typecheck, tests and production build to pass before merging.

High-level request flow

```
Student browser
   |  Firebase client SDK: sign-in + rule-checked reads/writes
   v
Next.js App Router (Vercel)
   |-- UI / Server Components
   |-- Route Handlers: validation, authorization, orchestration
   |-- Matching module: deterministic score + explanation
   |-- AI adapter: server-side Gemini call only
   v
Firebase Auth  ----  Cloud Firestore  ----  Cloud Storage (optional)
                          |
                 Security Rules + emulator tests

Production secrets: Vercel environment variables (never NEXT_PUBLIC_ for secrets)
AI provider failure: return baseline match result / a friendly retry state
```

Architectural guardrails

- Avoid two competing backend patterns. Use Next.js Route Handlers for synchronous MVP APIs; introduce Cloud Functions only when a specific scheduled or event-driven need appears.
- Separate client Firebase initialization from server-only Firebase Admin initialization. Never import service-account credentials into client bundles.
- Firebase Admin SDK server code bypasses Firestore Security Rules; enforce authorization and validate ownership again in each privileged server route.
- Keep provider/model names and prompts in one AI adapter, not scattered across UI components.
- Add an explicit feature flag for AI features. The product must remain usable if the flag is off or provider quota is exhausted.

6. AI matching, PulseIndex and AI mentor design

6.1 Start with a transparent scoring baseline

Do not train a recommender model at launch. You will not yet have enough reliable labels (successful collaborations, quality ratings or project outcomes) to train and validate one. Begin with a deterministic ranker, record feedback with consent, and evaluate it before adding learned ranking.

Feature	Initial weight	What it measures	Guardrail
Skill complementarity	35%	How much the candidate covers skills/roles explicitly requested by a team or project.	Reward needed skills; do not penalize a beginner for not listing every skill.
Goal / domain fit	25%	Overlap between self-declared project interests, domain tags and goals.	Show matching tags; allow users to correct them.

Feature	Initial weight	What it measures	Guardrail
Role coverage	20%	Whether the candidate fills a requested role (frontend, backend, AI/ML, design, research, etc.).	Use user-selected roles; do not infer role from demographic data.
Availability fit	10%	Overlap in stated availability or target event timeline.	Optional field; missing data is neutral, not an automatic penalty.
Collaboration preferences	10%	User-selected preferences such as async/sync, planned/flexible and communication preference.	Self-reported and editable only; never infer mental health or personality.

These weights are starting hypotheses, not scientific truth. Normalize each feature to 0–1, compute a weighted score, then calibrate against user feedback. Add missing-data handling explicitly; do not treat unknown as zero. Return 2–4 concise reasons tied to visible profile data.

Example response contract

```
type MatchSuggestion = {
  candidateId: string;
  score: number;                // 0..100, not a probability
  reasons: string[];            // grounded in visible profile/team fields
  matchedSkills: string[];
  complementarySkills: string[];
  algorithmVersion: string;     // e.g. "rules-v1"
};
```

6.2 Evaluation plan

- Create 30–50 synthetic or consented test profiles and 10–15 example team briefs. Include edge cases: missing skills, duplicate profiles, full teams, disabled accounts and sparse availability.
- Manually rate the top 5 suggestions for each brief as relevant / maybe / irrelevant, and record why. Keep the evaluation set versioned in tests/fixtures/matching/.
- Track Precision@5 (fraction of top five judged relevant), acceptance/request rate, accepted request rate, profile completion and user-reported match usefulness.
- Run an ablation: compare weights/features against the baseline before adding any new feature. Do not claim match quality is “AI-accurate” without measured evidence.
- Version the algorithm, weights and evaluation report. A changed score should be reproducible for the same inputs and version.

6.3 Embeddings and semantic match (later)

After a rules baseline works, test embeddings for semantic similarity between project descriptions, interests and skill summaries. Embeddings can help recognize related concepts, but they should not replace explicit role requirements or user controls. Evaluate against the same held-out manual relevance set and keep a deterministic fallback.

6.4 PulseIndex — keep it useful and fair

Position PulseIndex as a team/project pulse indicator, not a universal measure of a student’s worth. A first version can show profile readiness, role coverage, task status freshness and whether the team has a next milestone. Avoid private-message analysis, raw message counts, online-time surveillance or public “low performer” rankings.

6.5 AI mentor — safe minimum design

- Restrict the first mentor to project planning, breaking tasks down, explaining technical concepts and suggesting next steps.
- Ask for structured output (for example: summary, suggested steps, assumptions, links to verify) and validate against a schema before rendering.
- Do not automatically write AI output to a profile, send invitations, change membership or publish content. Require a human action for all consequential operations.
- Pass only the context necessary for the request. Do not send private chat history by default.
- Set per-user quotas, timeout/retry policy, maximum input length, output limits and a kill switch. Log latency, token/cost estimates and errors without logging secrets or unnecessary personal content.

7. Data model and security boundaries

Start with the smallest data model that supports the core journeys. Add indexes from actual query needs; document every composite index in source control if your Firebase setup supports it.

Collection / path	Core fields (illustrative)	Access principle
users/{uid}	displayName, photoURL, collegeId, year, createdAt, updatedAt, onboardingComplete	Private owner profile; only expose intentionally public fields.
publicProfiles/{uid}	headline, skills[], interests[], preferredRoles[], availability, links, updatedAt	Readable to authenticated users according to product visibility; owner controls edits.
teams/{teamId}	name, description, ownerUid, skillsNeeded[], rolesNeeded[], status, maxSize, createdAt	Visible to eligible signed-in users; edits limited to owner/admin.
teamMembers/{membershipId}	teamId, uid, role, status, joinedAt	Prevent duplicate active membership; only authorized team members/admin can update roles.
teamRequests/{requestId}	teamId, requesterUid, message, status, createdAt, reviewedAt	Requester creates/cancels own request; team owner/reviewer decides.
hackathons/{eventId}	title, organizer, url, deadline, startDate, format, location, tags[], sourceUrl, verifiedAt	Curated event data; edits limited to trusted maintainer/admin.
savedEvents/{uid_eventId}	uid, eventId, createdAt	User can only read/write own saves.
teams/{teamId}/tasks/{taskId}	title, ownerUid, status, dueDate, updatedAt	Visible to team members; update privileges follow team role.
conversations/{conversationId}/messages/{messageId}	senderUid, body, createdAt, reportStatus	Only members can read/write; enforce length/rate limits and report flow.
matchFeedback/{feedbackId}	fromUid, targetId, teamId?, outcome, reasonCode?, createdAt, algorithmVersion	Private, minimal and consented; only use for stated evaluation purpose.
reports/{reportId}	reporterUid, targetType, targetId, category, status, createdAt	Reporter can create/read own submission; moderation role handles review.

Security rules checklist

- Default deny. Add specific allow rules only after documenting who needs each read/write action.
- Test both allowed and denied cases using Firebase Emulator Suite or an equivalent isolated test project before deployment.
- Validate field allowlists and basic types in rules; repeat important validation on server routes.
- Do not trust a client-supplied uid, role, team membership, score or admin flag. Derive identity from verified auth and trusted database state.
- If server code uses Firebase Admin, verify the caller’s identity and authorization inside that server route because Admin access bypasses Firestore rules.
- Keep service-account private keys, Gemini keys and other privileged secrets in server-only environment variables. Do not commit .env.local.
- Add App Check where appropriate, abuse throttling for costly endpoints, upload limits for files, and data deletion/export handling.

Privacy rule of thumb

Collect only what makes a match or collaboration better. Make profile fields optional unless essential, provide clear visibility controls, and never use sensitive attributes as matching inputs.

8. Repository structure and setup commands

Use `pulsepoint` for the GitHub repository and package name. Keep the MVP in one repository to simplify shared contracts and deployment; create a separate AI service repo only if a distinct Python inference service later needs its own release lifecycle.

Recommended monorepo layout

```
pulsepoint/
├── .github/
│   ├── workflows/ci.yml
│   ├── ISSUE_TEMPLATE/bug_report.md
│   ├── ISSUE_TEMPLATE/feature_request.md
│   ├── pull_request_template.md
│   └── CODEOWNERS
├── docs/
│   ├── product/PRD.md
│   ├── architecture/system-overview.md
│   ├── architecture/data-model.md
│   ├── decisions/ADR-0001-mvp-stack.md
│   ├── ai/matching-v1.md
│   ├── ai/evaluation.md
│   └── runbooks/deploy-and-rollback.md
├── public/
│   └── brand/                  # logo and static assets
├── src/
│   ├── app/
│   │   ├── (marketing)/        # landing, about, public information
│   │   ├── (auth)/             # sign-in, sign-up
│   │   ├── (app)/              # dashboard, profile, teams, events
│   │   └── api/                # server route handlers
│   ├── components/
│   │   ├── ui/                 # generated/shared UI primitives
│   │   └── shared/             # nav, form fields, empty/error states
│   ├── features/
│   │   ├── auth/
│   │   ├── profiles/
│   │   ├── teams/
│   │   ├── hackathons/
│   │   ├── matching/
│   │   ├── collaboration/
│   │   └── moderation/
│   ├── lib/
│   │   ├── firebase/client.ts
│   │   ├── firebase/admin.ts   # server-only
│   │   ├── ai/gemini.ts        # server-only adapter
│   │   ├── validation/
│   │   ├── auth/
│   │   └── observability/
│   ├── types/
│   └── config/
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── e2e/
│   └── fixtures/matching/
├── firebase/
│   ├── firestore.rules
│   ├── firestore.indexes.json
│   └── storage.rules           # only if Storage is enabled
├── .env.example
├── .gitignore
├── package.json
├── README.md
├── tsconfig.json
└── next.config.ts
```

The exact route-group names may change during implementation. Preserve the boundaries: UI components should not contain ranking logic; feature modules should not expose secrets; server-only files should not be importable by client components.

8.3 Create the repository on Windows PowerShell

Prerequisites: a supported current Node.js LTS release, Git, a GitHub account, and Firebase access. Use the current create-next-app prompts; do not copy commands pinned to an outdated Next.js version.

```
# 1) Create the app
npx create-next-app@latest pulsepoint
cd pulsepoint

# 2) Add the shared UI / validation / Firebase SDK packages
npx shadcn@latest init
npm install firebase zod lucide-react

# 3) Add the current official Google GenAI SDK when ready for the AI phase
npm install @google/genai

# 4) Verify local baseline
npm run dev
# In a second terminal after adding scripts:
npm run lint
npx tsc --noEmit
npm run build
```

Command note: package APIs and scaffolding prompts can change. At setup time, follow the official current docs and commit the resulting lockfile. Add only packages that are actually used; avoid installing multiple competing AI SDKs.

8.4 Set up Git and the shared remote

```
git init
git add .
git commit -m "chore: bootstrap PulsePoint"
git branch -M main
git remote add origin https://github.com/<github-owner>/pulsepoint.git
git push -u origin main
```

Create the empty GitHub repository first, add both developers as collaborators, and confirm each person can clone it. Configure `main` protection before substantial feature work begins.

8.5 Environment variables

```
# .env.example — names only; never real values
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=
GEMINI_API_KEY=                 # server-side only; no NEXT_PUBLIC_ prefix
FIREBASE_ADMIN_PROJECT_ID=     # server-side only, if Admin SDK is used
FIREBASE_ADMIN_CLIENT_EMAIL=   # server-side only
FIREBASE_ADMIN_PRIVATE_KEY=    # server-side only; keep out of Git
```

Firebase web configuration values are intended to be used by the client SDK but must still be protected by Firebase Security Rules and App Check where appropriate. Privileged Admin credentials and the Gemini API key must never be exposed to the browser. Add `.env.local` to `.gitignore`; commit `.env.example` with names but no values. Use separate Firebase projects or clearly separated environments for development and production.

9. Git workflow, pull requests and collaboration rules

Branch strategy

Branch	Purpose	Rule
main	Always releasable / production baseline.	No direct feature pushes; merge via approved pull request and green checks.
feat/<issue>-short-name	New feature. Example: feat/42-team-requests	One logical change per branch; keep it short-lived.
fix/<issue>-short-name	Bug fix.	Add a regression test where practical.
docs/<topic>	Documentation only.	Still require review if it changes security, architecture or product commitments.
chore/<topic>	Tooling, dependencies, maintenance.	Call out lockfile/config changes explicitly.

Pull request checklist

- Issue linked; purpose and acceptance criteria stated.
- Screenshots or short screen recording for UI changes; request/response examples for API changes.
- `npm run lint`, `npx tsc --noEmit`, unit/integration tests and production build pass.
- Authorization checked on both allowed and denied paths; no secrets, personal datasets or `.env.local` in the diff.
- Database/schema/rules/index changes and migration or rollout notes documented.
- Failure, loading, empty and unauthorized states considered.
- Reviewer approves; preview deployment tested for the critical journey; merge only after required checks pass.

Commit format

```
feat(matching): add skill-complementarity score
fix(teams): prevent duplicate active membership
test(matching): cover missing-availability case
docs(ai): document rules-v1 weights
chore(ci): run typecheck on pull requests
```

Branch protection and CI

- Protect `main`: require pull request, all conversations resolved, and required CI checks. Where supported, require one approving review; with two people, the author should not approve their own PR.
- CI jobs: install from lockfile, lint, typecheck, unit tests, targeted integration tests, and production build.
- Run Firebase Emulator tests for rules and server authorization where practical. Do not point CI at production data.
- Use Dependabot or periodic dependency review, but update dependencies in focused PRs and run the full test suite.
- Use Code Owners for `firebase/`, `.github/workflows/`, `src/lib/firebase/`, `src/lib/ai/` and deployment/runbook files.

Two-person review reality

Because there are only two of you, some tiny documentation or emergency fixes may need a lightweight path. Agree on that exception in advance; never bypass checks for auth, access-control, secret or production-data changes.

10. Testing, observability and deployment

Testing pyramid

Level	What to test	Examples
Unit	Pure logic with no Firebase or network.	Scoring weights, normalization, missing-data handling, role filters, form validators.
Integration	Boundaries between app code and services using emulators/mocks.	Firestore rules, team membership writes, request acceptance, server authorization, AI response schema validation.
End-to-end	Real user journeys in a test environment.	Sign up → finish profile → create team → request teammate → owner accepts → team page loads.
AI quality	Semantic correctness and policy adherence.	Match reasons grounded in profile fields, schema compliance, prompt injection, unsupported advice and empty context.
Release smoke	Production-critical paths and observability.	Sign-in, profile read/write, create/request team, match response fallback, event URL, report creation.

Required quality gates

- No secrets in source control or browser bundles. Add secret scanning if available.
- Authorization tests cover user A trying to read or change user B's private data, membership and reports.
- AI responses are validated and treated as untrusted content; do not render raw HTML from model output.
- A failed or rate-limited AI request returns a useful fallback, not a broken dashboard.
- Critical pages have accessible labels, keyboard navigation, mobile layout and meaningful empty/error states.
- Log errors with request/correlation IDs but redact tokens, private message bodies and unnecessary personal information.

Environments and deployment path

Environment	Purpose	Rules
Local	Developer machine, emulator or development Firebase project.	`.env.local` is ignored by Git; no production secrets/data.
Preview	Per-branch/PR Vercel deployment.	Use preview-scoped variables and test with a non-production Firebase project where feasible.
Production	User-facing application.	Only deploy merged, green, reviewed changes; verify smoke test and rollback path.

Vercel supports distinct local, preview and production environments. Treat environment variables as deployment configuration: changes generally require a new deployment to take effect. Keep the operational checklist in `docs/runbooks/deploy-and-rollback.md`.

Minimal monitoring

- Application errors by route and release; build/deploy status.
- AI endpoint success/error rate, latency, estimated usage and rate-limit events.
- Core funnel: sign-up → profile completion → first match viewed → team request sent → request accepted.
- Report/moderation backlog and time to first review.
- A weekly 20-minute review of top user friction and the highest-severity defects.

11. Privacy, safety, risks and mitigation

Privacy and trust baseline

- Publish a plain-language privacy notice before inviting beta users; explain what is public, what is private and how matching works.
- Collect only fields with a clear product purpose; allow optional availability and collaboration-preference fields.
- Provide profile visibility controls, blocking/reporting and a deletion workflow. Decide retention periods for messages, feedback and reports.
- Do not use caste, religion, health, sexuality, political views or other sensitive attributes for ranking. Do not infer them from names, photos, chat or college details.
- Do not claim to detect motivation, personality, honesty or compatibility as a fact. Describe recommendations as suggestions based on selected fields.
- For hackathon listings, show source URL and `verifiedAt`; avoid implying the app is the organizer or guarantees eligibility/deadlines.
- Set AI quotas, input/output size limits and basic abuse controls before public access.

Risk register

Risk	Probability / impact	Mitigation	Owner
Scope creep from too many AI features	High / High	Lock MVP; use explicit cut line; every new feature must replace another item or wait.	Both
Matching suggestions feel random	Medium / High	Explain score, test on manual relevance set, collect reason-coded feedback, version ranking.	AI/ML
Sensitive data exposure / insecure rules	Medium / Critical	Default-deny rules, emulator tests, server auth checks, review all rule/admin changes, no production data in CI.	Full-stack + both review
Gemini quota, latency or model/API change	Medium / Medium	Central adapter, schema validation, rate limits, timeouts, kill switch and non-AI fallback.	AI/ML + full-stack
Event listings become stale	High / Medium	Curated source, last-verified date, event owner, report correction workflow.	Full-stack / shared ops
Uneven contribution / blocked PRs	Medium / High	One owner per issue, small PRs, explicit handoffs, weekly demo, documented decisions.	Both

Risk	Probability / impact	Mitigation	Owner
Cost grows unexpectedly	Medium / Medium	Budget alerts where available, per-user quotas, AI only on user action, cache safe repeat work.	AI/ML + full-stack
Poor early user retention	Medium / High	Interview users, measure first-value time, prioritize relevant matches and completed team formation over vanity metrics.	Both

12. Launch criteria, metrics and next actions

Beta launch checklist

- At least 10–20 invited students with varied profiles; do not rely on founder-only test accounts.
- Fresh-account smoke test passes from sign-in to profile to matching to team request/acceptance.
- Security rules and unauthorized access tests pass in the intended test environment.
- Match explanations cite visible fields; empty and low-confidence cases show honest messaging.
- Event listings have official source links and verified dates; report/correction process works.
- Privacy notice, terms/basic community guidelines, contact/support path and account deletion process are present.
- AI feature can be disabled and the core workflow still works.
- Both founders know how to identify a bad deployment, review logs and roll back.

Metrics worth tracking

Metric	Definition	Use
Activation	Share of sign-ups completing the minimum profile and viewing one match or team.	Find onboarding friction.
Time to first value	Time from sign-up to first useful match/team/event action.	Prioritize the shortest useful path.
Match relevance	Share of top-5 suggestions rated relevant by user review or agreed evaluation rubric.	Tune ranking and explanations.
Team formation	Requests accepted and teams reaching minimum viable membership.	Check whether recommendations cause outcomes.
Repeat usage	Users returning for a second meaningful session in 7 days.	Assess repeat value, not page views alone.
Reliability	Critical route error rate, p95 response latency where available, deployment failure rate.	Protect core experience.
Trust	Reports reviewed, account deletion completed, privacy/support complaints.	Catch safety and trust problems early.

First five actions after reading

1. Hold a 60–90 minute founder kickoff and agree on MVP, schedule capacity, weekly cadence and who has final say when there is a tie.
2. Interview 5–8 students and validate that teammate finding is a real, repeated pain—not just a feature people say sounds cool.
3. Create `pulsepoint` on GitHub, add both collaborators, protect `main`, and make CI a merge gate.
4. Write the MVP PRD, user flows, first Firestore data model and matching-v1 scoring document under `docs/`.
5. Deploy the smallest skeleton app with auth placeholder, CI and preview deployment before implementing complex features.

Success definition for the first release

A student can create a useful profile, discover a relevant hackathon or team, understand why a teammate is suggested, send a request, and coordinate the next step—with working privacy/security controls and without depending on AI being online.

Appendix A. Definition of Done and acceptance checklist

Use this on every issue. A feature is not done simply because it works on the author's machine.

- Acceptance criteria written and met; edge cases and error states handled.
- Input validated at runtime; authorization checked at the correct trust boundary.
- Loading, empty, error and unauthorized states are understandable.
- Unit/integration/E2E coverage added where risk warrants; tests pass locally and in CI.
- UI checked on a narrow/mobile viewport and basic keyboard navigation.
- No secrets, personally identifiable test data or unrelated formatting changes included.
- Docs, environment example, rules, indexes and runbook updated if behavior changed.
- Small pull request reviewed and tested on Preview where relevant.
- Monitoring, fallback, rollback or support approach considered for risky features.

Suggested GitHub Issues to create in Week 1

Epic	Initial issues
EPIC-01 Product definition	Interview notes; PRD; target user flows; MVP/non-goals; acceptance criteria.
EPIC-02 Repo and CI	Bootstrap app; lint/typecheck/test/build scripts; GitHub Actions; branch protection; PR template.
EPIC-03 Identity and profiles	Firebase project; auth; profile schema; onboarding; visibility; rules/emulator tests.
EPIC-04 Teams	Team schema; create team; member roles; join request; accept/reject; duplicate prevention.
EPIC-05 Matching v1	Feature schema; weighted ranker; reasons; fixture set; ranking evaluation report; feedback capture.
EPIC-06 Hackathon discovery	Event schema; seed/admin flow; search filters; bookmarks; source/verification date.
EPIC-07 Collaboration	Team dashboard; task status; basic messaging decision; report/block workflow.
EPIC-08 AI assist	Gemini adapter; schema validation; quota/rate limit; quality tests; kill switch/fallback.
EPIC-09 Beta and launch	Privacy notice; E2E test; beta feedback sessions; release checklist; monitoring; retrospective.

Appendix B. Official technical references

The stack recommendation was cross-checked against official documentation available on 29 October 2026. Re-check model names, package APIs and quotas when implementation begins; provider capabilities and rate limits evolve.

Next.js App Router — https://nextjs.org/docs/app Routing, Server Components and App Router concepts.

Next.js project structure — https://nextjs.org/docs/pages/getting-started/project-structure Folders, file conventions and `.env` handling; route choice should follow current App Router docs.

Next.js environment variables — https://nextjs.org/docs/pages/guides/environment-variables `NEXT_PUBLIC_` variables are bundled for browser use; keep secrets server-side.

Firebase Security Rules overview — https://firebase.google.com/docs/rules Rules model and testing approach.

Firestore security overview — https://firebase.google.com/docs/firestore/security/overview Client SDK access should use Authentication and Security Rules; server libraries require IAM/server controls.

Firestore rule conditions — https://firebase.google.com/docs/firestore/security/rules-conditions Authentication and user-owned data rule patterns.

Gemini structured outputs — https://ai.google.dev/gemini-api/docs/structured-output Schema-constrained output and the need to validate semantic correctness.

Gemini embeddings — https://ai.google.dev/gemini-api/docs/embeddings Embedding-based similarity concepts, task types and model documentation.

Gemini API rate limits — https://ai.google.dev/gemini-api/docs/rate-limits Limits vary by model/tier; review active quotas in AI Studio before launch.

Vercel environments — https://vercel.com/docs/deployments/environments Local, Preview and Production environment model.

Vercel environment variables — https://vercel.com/docs/environment-variables Environment-scoped values and redeployment behavior.

GitHub protected branches — https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches PR review and status-check protections.

GitHub pull request and CODEOWNERS guidance — https://docs.github.com/en/pull-requests/reference/managing-and-standardizing-pull-requests Templates and code ownership practices.

Final founder agreement

Before writing feature code, both founders should explicitly agree to these defaults:

- The MVP is the core student → profile → discover → match → request/join → collaborate loop.
- The initial matching score is a transparent rule-based heuristic, not a probability or claim of objective compatibility.
- The full-stack founder owns production deployment and the main app architecture; the AI/ML founder owns ranking evaluation and AI quality. Both review security-sensitive changes.
- No direct push to `main`, no unreviewed production changes, no secrets in Git, and no AI-only critical path.
- Every new feature has an owner, acceptance criteria, and a clear reason it matters to the student.